

LEAD DEVELOPER TECHNICAL REFERENCE

KaamMilega™ Flutter Application

Complete Architecture, State Management, Real-Time Chat, API Integration & Data Flow Tracing

Project:	KaamMilega Mobile App (Flutter Client)
Backend Target:	Go Fiber v2 + Uber Fx + MongoDB (km-backend)
Frameworks:	Flutter 3.x, Flutter Riverpod 3.4, GoRouter 18.0, Dio 5.11
Primary Scope:	Clean Architecture, 5-Tab Navigation, Real-Time WebSockets, Auth, Jobs & Applications
Generated Date:	September 2026

KaamMilega Flutter Application — Complete Codebase Analysis & Developer Documentation

1. Project Overview

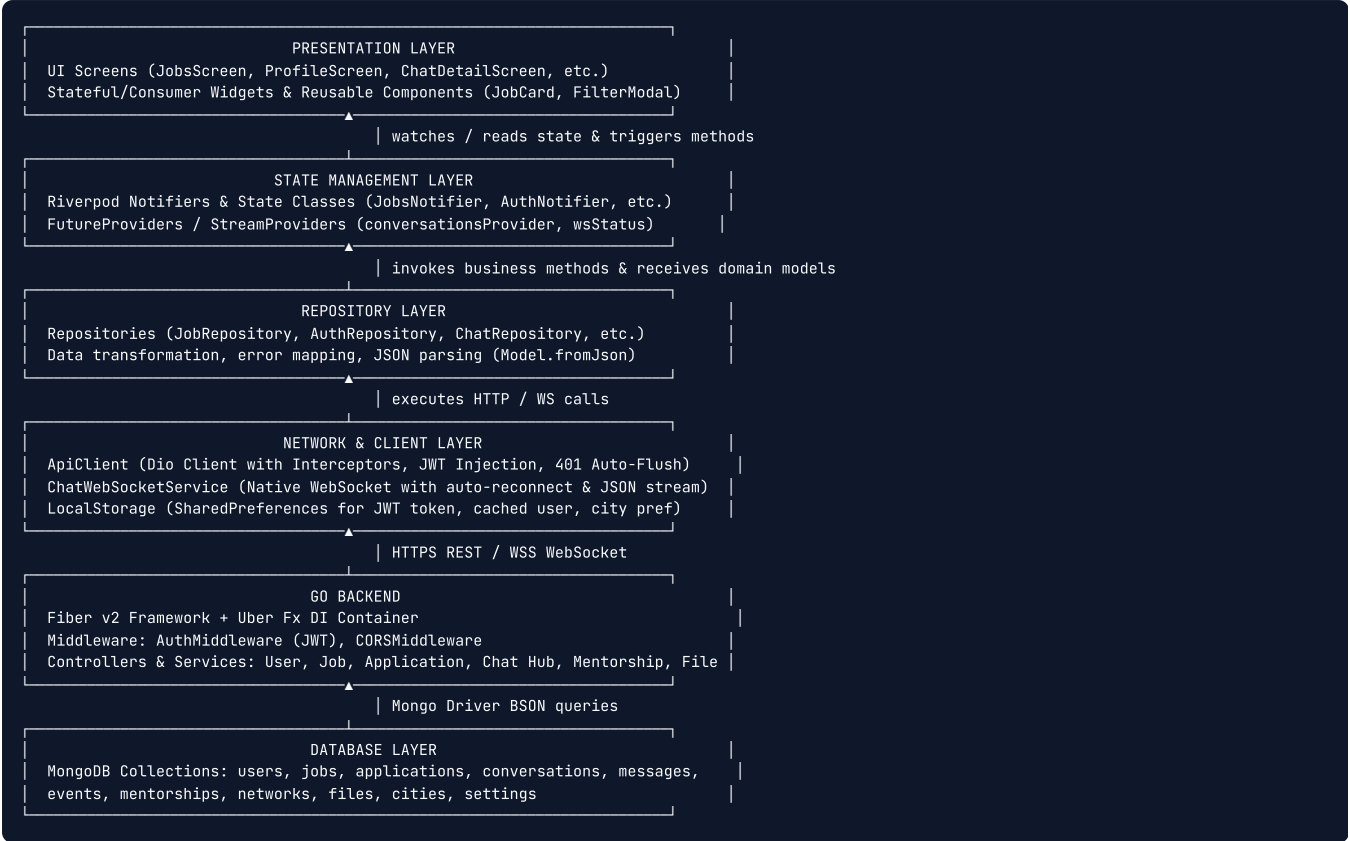
- **Application Name:** KaamMilega™ (Mobile Client)
- **Application Package / Identifier:** name: kaam_milega (Version: 1.0.0+1)
- **Primary Target Audience:** Blue-collar, grey-collar, gig-workers, skilled tradespeople, and jobseekers across India (supporting Hindi & English localization contexts).
- **Architecture Style:** Feature-First / Layered Architecture with **Flutter Riverpod (v3.4.3)**, **GoRouter (v18.0.1)**, and **Dio (v5.11.1)** communicating via REST & WebSockets with a modular **Go (Fiber v2 + Uber Fx + MongoDB)** backend.
- **Workspace Repository Path:** D:\Desktop\Kaammilega App\KaamMilegaApp
- **Go Backend Repository Path:** D:\Desktop\Kaammilega App\KaamMilega\km-backend
- **Web Portal Repository Path:** D:\Desktop\Kaammilega App\KaamMilega\km-frontend

2. Project Structure & Directory Breakdown

```
D:\Desktop\Kaammilega App\KaamMilegaApp\
├─ pubspec.yaml           # App dependencies, SDK version (^3.13.3), asset declarations
├─ analysis_options.yaml  # Static analysis and linter rules (flutter_lints ^6.0.0)
├─ lib/
│  ├─ main.dart           # App entry point, WidgetsFlutterBinding, LocalStorage initialization
│  └─ app/
│     ├─ app.dart         # Root KaamMilegaApp widget configuring MaterialApp.router & theme
│     ├─ router.dart      # Central GoRouter declarative route definitions (30+ routes)
│     └─ theme/
│        ├─ app_colors.dart # Semantic color tokens (primary, accent, deepNavy, surface, status)
│        ├─ app_text_styles.dart # Typography scale and weights
│        └─ theme.dart     # ThemeData definition (Material 3, light theme, inputs, buttons)
├─ core/
│  ├─ constants/
│  │  ├─ api_constants.dart # Backend REST/WS endpoints, image resolver utility
│  │  └─ app_branding.dart  # Branding assets, logotype, and static text constants
│  ├─ network/
│  │  └─ api_client.dart    # Central Dio wrapper, interceptors (Bearer token, 401 handling, errors)
│  └─ storage/
│     └─ local_storage.dart # SharedPreferences persistence (JWT, user profile, saved jobs, city)
├─ features/
│  ├─ auth/                # Authentication (OTP, Email/Password, Registration, Password Reset)
│  ├─ profile/             # Candidate Profile, Experience, Education, Projects, Resume, Drawer
│  ├─ jobs/                # Jobs Discovery, Search, Multi-parameter Filters, Saved Jobs, Pagination
│  ├─ applications/        # Job Application Submission, Apply Sheet, Tracking Statuses
│  ├─ instant_work/        # Gig-worker spot dispatch & on-demand matching
│  ├─ network/             # P2P Candidate & Recruiter Connections, Invites, Status
│  ├─ chat/                # Real-time WebSocket Messaging, Conversation List, Chat Room
│  ├─ wallet/              # Ledger Transactions, Balance, Add Money, Payouts, P2P Transfers
│  ├─ experts/             # 1-on-1 Mentorship & Industry Expert Booking
│  ├─ events/              # Career Fairs, Webinars, Event Registration
│  ├─ skills_marketplace/  # Trade & Skill Catalog, Search, Categories
│  ├─ interviews/          # Candidate Interview Schedules & Statuses
│  ├─ notifications/       # Activity Feeds, Push Notification Alerts
│  ├─ cities/              # Pan-India Location Selector & Filter Sheet
│  ├─ company/             # Verified Employer Company Hub & Profile
│  ├─ explore/             # Category Discovery Matrix
│  ├─ feed/                # Community Updates, Career Advice, Posts
│  ├─ peer_to_peer/        # Peer Skill Exchange Hub
│  ├─ services/            # Blue-collar Local Services Marketplace
│  ├─ splash/              # Splash Screen & Session Verification Router
│  └─ navigation/          # 5-Tab Scaffold Shell (Home, Jobs, Center [+], Chats, Profile)
├─ shared/
│  └─ widgets/             # Reusable UI Atoms & Molecules (Buttons, Shimmers, Inputs, Sheets)
```

3. Complete Project Architecture Explanation

The KaamMilega application implements a **Clean, Feature-Driven Layered Architecture**:



Layer Responsibilities & Exact Locations

Layer	Responsibility	Key Files
Presentation	Renders UI, listens to Riverpod providers via <code>ref.watch()</code> , emits user events via <code>ref.read().notifier</code> .	<code>lib/features/**/presentation/*.dart</code> , <code>lib/shared/widgets/*.dart</code>
State Management	Holds immutable state classes (<code>JobsState</code> , <code>AuthState</code>), implements state mutation logic, handles background fetching and error states.	<code>lib/features/**/providers/*_provider.dart</code>
Repository	Mediates between UI state and raw network APIs; serializes Dart domain models from backend JSON.	<code>lib/features/**/repositories/*_repository.dart</code>
Network & Storage	Manages network transport (Dio), JWT header injection, response error parsing, connection lifecycle, and local disk caching.	<code>lib/core/network/api_client.dart</code> , <code>lib/core/storage/local_storage.dart</code> , <code>lib/features/chat/services/chat_websocket_service.dart</code>
Backend API	Exposes Fiber routes, verifies JWT signatures, enforces role checks, executes business logic, and coordinates database mutations.	<code>km-backend/cmd/main.go</code> , <code>km-backend/internal/features/**/*.go</code>

4. Application Startup & Navigation Flow

Execution Sequence on App Launch:

```
main() [lib/main.dart]
├─ 1. WidgetsFlutterBinding.ensureInitialized()
├─ 2. LocalStorage.init() (Pre-warms SharedPreferences)
├─ 3. runApp(ProviderScope(child: KaamMilegaApp()))
    └─ MaterialApp.router [lib/app/app.dart]
        └─ AppRouter.router [lib/app/router.dart]
            └─ Initial Location: '/splash'
                └─ SplashScreen [lib/features/splash/presentation/splash_screen.dart]
                    └─ Calls ref.read(authProvider.notifier).checkAuthStatus()
                        └─ Reads LocalStorage.getToken()
                            └─ Token present → Refreshes profile via GET /api/user/profile
                                └─ Token absent → Guest session initialized
                                    └─ Timer(2.2s) → context.go('/home')
```

Central Navigation Route Map (`lib/app/router.dart`)

```

/splash          → SplashScreen
/home            → MainNavigationShell(initialIndex: 0) [HomeScreen]
/jobs            → MainNavigationShell(initialIndex: 1) [JobsScreen]
/chats           → MainNavigationShell(initialIndex: 3) [ChatListScreen]
/profile         → MainNavigationShell(initialIndex: 4) [ProfileScreen]
/jobs/:id        → JobDetailScreen(jobId: :id)
/saved-jobs      → SavedJobsScreen
/my-applications → MyApplicationsScreen
/applications/:id → ApplicationDetailScreen(applicationId: :id)
/chats/:id       → ChatDetailScreen(conversationId: :id, receiverId, title)
/login           → LoginScreen
/register        → RegisterScreen
/otp             → OtpScreen(phone)
/forgot-password → ForgotPasswordScreen(initialEmail)
/settings        → SettingsScreen
/wallet          → WalletScreen
/wallet/transactions → WalletTransactionsScreen
/wallet/add-money → WalletAddMoneyScreen
/wallet/withdraw → WalletWithdrawScreen
/wallet/transfer → WalletTransferScreen
/explore         → ExploreScreen
/instant-work    → InstantWorkScreen
/skills-marketplace → SkillsMarketplaceScreen
/experts         → ExpertsScreen
/apply-expert    → ApplyExpertScreen
/services        → ServicesMarketplaceScreen
/peer-to-peer    → PeerToPeerScreen
/events          → EventsScreen
/notifications   → NotificationsScreen
/company/:id     → CompanyScreen(companyId: :id)
/interviews      → InterviewsScreen
/network         → NetworkScreen
/feed           → FeedScreen

```

5. Major Feature Implementations

Feature 1: Authentication & Session Management

1. Architecture Flow

```

LoginScreen / RegisterScreen / OtpScreen
  ↓ (User enters credentials / phone)
ref.read(authProvider.notifier).loginWithPassword() / sendOtp() / verifyOtp()
  ↓
AuthRepository [lib/features/auth/repositories/auth_repository.dart]
  ↓
ApiClient.post(ApiConstants.LoginPassword / verifyOtp)
  ↓
HTTP POST https://api.kaammilega.com/api/auth/login/password
  ↓
Go Controller: UserController.LoginWithPassword [km-backend/internal/features/user/controller.go]
  ↓
Go Service & Mongo DB: Verifies bcrypt password hash / OTP in Redis/Memory
  ↓
Go Response: { "token": "eyJhbGciOi...", "user": { ... }, "is_registered": true }
  ↓
AuthRepository: LocalStorage.saveToken(token) & LocalStorage.saveUser(user)
  ↓
AuthNotifier: state = state.copyWith(isAuthenticated: true, user: user)
  ↓
GoRouter / UI redirects or refreshes profile in-place

```

2. Key Files & Roles

- `lib/features/auth/presentation/login_screen.dart` : Dual-tab login UI allowing either Mobile OTP Login or Email/Password Login with form validation and password visibility toggling.
- `lib/features/auth/presentation/otp_screen.dart` : 6–digit PIN input with resend countdown timer and automatic dispatch to `verifyOtp`.
- `lib/features/auth/presentation/register_screen.dart` : Full registration flow capturing candidate name, email, phone, and password.
- `lib/features/auth/presentation/forgot_password_screen.dart` : 2–step password recovery calling `POST /api/auth/password/forgot` and `POST /api/auth/password/reset`.
- `lib/features/auth/repositories/auth_repository.dart` : Handles token persistence, login requests, user registration, and profile updates.
- `lib/features/auth/providers/auth_provider.dart` : Exposes `authProvider` (`NotifierProvider<AuthNotifier, AuthState>`) managing global authentication state.

Feature 2: Candidate Profile & Experience System

1. Architecture Flow

```

ProfileScreen / ProfileDrawer
  ↓ (Watches authProvider.user)
User edits Bio / Adds Experience / Adds Education / Adds Project / Uploads Resume
  ↓
AuthNotifier.updateProfile() / addExperience() / addEducation() / uploadResumePdf()
  ↓
AuthRepository
  ↓
Multipart File Upload: POST /api/files/upload → Returns CDN / Static URL
Profile Record Patch: PATCH /api/user/profile OR POST /api/user/experience
  ↓
Go Backend: UserController.UpdateProfile / AddExperience → MongoDB `users` collection update
  ↓
Updated UserProfile returned → Saved to LocalStorage → AuthState updated → UI re-renders

```

2. Data Model (lib/features/auth/models/user_profile.dart)

- `id` : MongoDB Object ID.
- `name` , `mobile` , `email` , `headline` , `about` , `city` , `state` .
- `roles` : List of user roles (defaults to `['user']`).
- `education` : List of `EducationItem` (`schoolName` , `degree` , `fieldOfStudy` , `startDate` , `endDate` , `grade`).
- `experience` : List of `ExperienceItem` (`title` , `companyName` , `employmentType` , `location` , `startDate` , `endDate` , `description`).
- `projects` : List of `ProjectItem` (`title` , `associatedWith` , `description` , `link` , `startDate` , `endDate` , `skills`).
- `skills` : List of skill strings.
- `profileImage` , `coverImage` , `resumeUrl` : Normalized via `ApiConstants.resolveImageUrl()` .
- `walletBalance` , `profileViewsCount` , `postImpressionsCount` , `searchAppearancesCount` .

Feature 3: Jobs Discovery, Filtering & Search

1. Architecture Flow

```

JobsScreen [lib/features/jobs/presentation/jobs_screen.dart]
  ↓
ref.watch(jobsProvider) → JobsState(jobs, totalJobs, isLoading, filter)
  ↓ (User types query, selects city, or toggles filter in FilterModal)
JobsNotifier.setSearchQuery() / setCity() / applyFilter() [lib/features/jobs/providers/jobs_provider.dart]
  ↓
JobRepository.getJobs(filter) [lib/features/jobs/repositories/job_repository.dart]
  ↓
ApiClient.get(ApiConstants.jobs, queryParameters: filter.toQueryParams())
  ↓
HTTP GET https://api.kaammilega.com/api/jobs?page=1&limit=10&city_ids=Mumbai&job_types=Full-time&salary_min=20000
  ↓
Go Backend: JobController.GetJobs [km-backend/internal/features/job/controller.go]
  ↓
MongoDB BSON Filter Query executed against `jobs` collection
  ↓
Response JSON: { "jobs": [ { ... } ], "total": 48, "page": 1, "limit": 10 }
  ↓
JobsResponse.fromJson → JobsState updated → JobsScreen displays JobCard list

```

2. Filtering Capabilities (lib/features/jobs/models/job_filter.dart)

- `searchQuery` : Full-text substring search across title, description, and company name.
- `city` : Filter by target city ID or name.
- `jobTypes` : Multi-select (`Full-time` , `Part-time` , `Contract` , `Internship`).
- `salaryRange` : Minimum threshold (`₹5,000` , `₹10,000` , `₹20,000` , `₹30,000+`).
- `experience` : Experience brackets (`Fresher` , `1-3 Yrs` , `4-5 Yrs` , `5+ Yrs`).
- `genders` : Recruiter preference filter (`Male` , `Female` , `Any`).
- `qualification` : Education requirements (`10th Pass` , `12th Pass` , `Graduate` , `Post Graduate`).

Feature 4: Job Application & Candidate Tracking

1. Architecture Flow

```

JobDetailScreen → User clicks "Apply Now"
↓
ApplyModal.show(context, job) [lib/features/applications/presentation/apply_modal.dart]
↓ (User enters cover letter / notes and confirms application)
ApplicationRepository.applyToJob(jobId, coverLetter) [lib/features/applications/repositories/application_repository.dart]
↓
ApiClient.post(ApiConstants.applications, data: { 'job_id': jobId, 'cover_letter': coverLetter })
↓
HTTP POST https://api.kaammilega.com/api/applications (Authorization: Bearer <token>)
↓
Go Backend: ApplicationController.CreateApplication [km-backend/internal/features/application/controller.go]
├─ Checks for duplicate application: Repo.FindByJobAndCandidate() → Returns 400 if exists
└─ Inserts application record with status = "Applied" into 'applications' collection
↓
Success → UI displays confirmation toast → Invalidates myApplicationsProvider

```

2. Candidate Application Screen (lib/features/applications/presentation/my_applications_screen.dart)

- Displays active and historical applications with status badges (Applied , Reviewing , Shortlisted , Interview Scheduled , Rejected , Hired).
- Tapping an application navigates to ApplicationDetailScreen displaying interview links and recruiter feedback.

Feature 5: Real-Time Chat & WebSocket Messaging

1. Dual REST + WebSocket Architecture

```

REST API FLOW (Initial History & Sending Message):
User opens Chat → ChatMessagesNotifier → ChatRepository.getMessage(conversationId)
├─ GET /api/chats/:id/messages → Returns historical message array

User sends message → ChatRepository.sendMessage(receiverId, content)
├─ POST /api/chats/messages → Saves to MongoDB 'messages' collection

REAL-TIME WEBSOCKET FLOW (Instant Live Updates):
App connects → ChatWebSocketService.connect()
├─ wss://api.kaammilega.com/api/ws/chats?token=<JWT_TOKEN>
↓
Go Backend: ChatApi.WebSocketHandler [km-backend/internal/features/chat/controller.go]
├─ Hub.Register(userID, conn) [km-backend/internal/features/chat/hub.go]

When Message Arrives:
Go Controller broadcasts { "type": "NEW_MESSAGE", "message": { ... } } via Hub.Send(receiverID)
↓
ChatWebSocketService._onMessage receives JSON frame
↓
Broadcasted over messageStream
↓
ChatMessagesNotifier listens → Appends message without duplicate IDs → UI re-renders instantly

```

2. Resilience & Error Handling in ChatWebSocketService

- Heartbeat & Reconnection:** Automatically attempts exponential reconnection (2s, 4s, 6s, 8s, 10s up to 5 attempts) on network drop or socket close.
- Deduplication:** Maintains _processedMessageIds set preventing duplicate bubble renders between optimistic POST responses and incoming WebSocket frames.

Feature 6: Networking & Peer-to-Peer Connections

1. Architecture Flow

```

NetworkScreen [lib/features/network/presentation/network_screen.dart]
├─ pendingInvitationsProvider → GET /api/network/pending
├─ connectionsProvider → GET /api/network/connections

Actions:
├─ "Connect" → NetworkRepository.sendInvitation(userID) → POST /api/network/connect
├─ "Accept" → NetworkRepository.acceptInvitation(userID) → POST /api/network/accept
├─ "Ignore" → NetworkRepository.ignoreInvitation(userID) → POST /api/network/ignore
├─ "Remove" → NetworkRepository.deleteConnection(userID) → DELETE /api/network/connections/:id

```

Feature 7: Digital Wallet & Ledger System

1. Architecture Flow

```
WalletScreen [lib/features/wallet/presentation/wallet_screen.dart]
├─ Balance: Derived from authProvider.user.walletBalance
├─ Transactions: walletTransactionsProvider → GET /api/wallet/transactions

Modules:
├─ Add Money: WalletAddMoneyScreen → POST /api/wallet/add-money
├─ Withdraw: WalletWithdrawScreen → POST /api/wallet/withdraw
├─ Transfer: WalletTransferScreen → POST /api/wallet/transfer (P2P identifier & note)
```

Feature 8: Experts, Mentorship & Events

- **Experts / Mentorship (lib/features/experts/):**
 - ExpertRepository.getExperts() calls GET /api/mentorships with optional category filtering.
 - ExpertRepository.bookSession() calls POST /api/mentorships/book with mentorship ID and scheduled timestamp.
- **Events & Webinars (lib/features/events/):**
 - EventRepository.getEvents() calls GET /api/events with pagination and search.
 - EventRepository.registerForEvent() calls POST /api/events/:id/register to secure candidate attendance.

6. Central API Endpoint Inventory

Feature	Method	Endpoint	Flutter File	Repository Method	Provider / Notifier
Auth	POST	/api/auth/otp/send	auth_repository.dart	sendOtp(mobile)	authProvider.notifier
Auth	POST	/api/auth/otp/verify	auth_repository.dart	verifyOtp(mobile, code)	authProvider.notifier
Auth	POST	/api/auth/login/password	auth_repository.dart	loginWithPassword(email, pwd)	authProvider.notifier
Auth	POST	/api/auth/register/password	auth_repository.dart	registerWithPassword(...)	authProvider.notifier
Auth	POST	/api/auth/password/forgot	auth_repository.dart	forgotPassword(email)	authProvider.notifier
Auth	POST	/api/auth/password/reset	auth_repository.dart	resetPassword(...)	authProvider.notifier
Auth	POST	/api/auth/otp/email/send	auth_repository.dart	sendEmailOtp(email)	authProvider.notifier
Auth	POST	/api/auth/otp/email/verify	auth_repository.dart	verifyEmailOtp(email, otp)	authProvider.notifier
User	GET	/api/user/profile	auth_repository.dart	getProfile()	authProvider.notifier
User	PATCH	/api/user/profile	auth_repository.dart	updateProfile(map)	authProvider.notifier
User	POST	/api/user/education	auth_repository.dart	addEducation(...)	authProvider.notifier
User	POST	/api/user/experience	auth_repository.dart	addExperience(...)	authProvider.notifier
User	POST	/api/user/project	auth_repository.dart	addProject(...)	authProvider.notifier
User	POST	/api/user/skill	auth_repository.dart	addSkill(name)	authProvider.notifier
Files	POST	/api/files/upload	auth_repository.dart	uploadFile(bytes, name)	authProvider.notifier
Jobs	GET	/api/jobs	job_repository.dart	getJobs(filter)	jobsProvider.notifier
Jobs	GET	/api/jobs/:id	job_repository.dart	getJobById(id)	Future call
Applications	POST	/api/applications	application_repository.dart	applyToJob(jobId, letter)	Direct method call
Applications	GET	/api/applications/my	application_repository.dart	getMyApplications()	myApplicationsProvider
Interviews	GET	/api/interviews/my	interview_repository.dart	getMyInterviews()	myInterviewsProvider
Cities	GET	/api/cities	city_repository.dart	getCities()	citiesProvider

Feature	Method	Endpoint	Flutter File	Repository Method	Provider / Notifier
Network	POST	/api/network/connect	network_repository.dart	sendInvitation(id)	Direct method call
Network	POST	/api/network/accept	network_repository.dart	acceptInvitation(id)	Direct method call
Network	POST	/api/network/ignore	network_repository.dart	ignoreInvitation(id)	Direct method call
Network	GET	/api/network/pending	network_repository.dart	getPendingInvitations()	pendingInvitationsProvider
Network	GET	/api/network/connections	network_repository.dart	getConnections()	connectionsProvider
Network	GET	/api/network/status/:id	network_repository.dart	getConnectionStatus(id)	connectionStatusProvider
Chat	GET	/api/chats	chat_repository.dart	getConversations()	conversationsProvider
Chat	POST	/api/chats/messages	chat_repository.dart	sendMessage(...)	chatMessagesProvider
Chat	GET	/api/chats/:id/messages	chat_repository.dart	getMessages(id)	chatMessagesProvider
Chat WS	WS	/api/ws/chats?token=...	chat_websocket_service.dart	connect()	websocketStatusStreamProvider
Events	GET	/api/events	event_repository.dart	getEvents(...)	eventsProvider
Events	POST	/api/events/:id/register	event_repository.dart	registerForEvent(id)	Direct method call
Mentorship	GET	/api/mentorships	expert_repository.dart	getExperts(...)	expertsProvider
Mentorship	POST	/api/mentorships/book	expert_repository.dart	bookSession(...)	Direct method call
Skills	GET	/api/skills	skills_repository.dart	getSkills(...)	skillsProvider
Notifications	GET	/api/notifications	notification_repository.dart	getNotifications()	notificationsProvider

7. Backend & Database Architecture

Go Backend Structure (km-backend)

- **Framework:** Go Fiber v2 (github.com/gofiber/fiber/v2)
- **Dependency Injection:** Uber Fx (go.uber.org/fx)
- **Database Driver:** Official MongoDB Go Driver (go.mongodb.org/mongo-driver/mongo)
- **Authentication Middleware:** JWT parsing via github.com/golang-jwt/jwt/v5 checking the `sub` claim for the user's MongoDB `_id`.

```
km-backend/
├── cmd/main.go           # Fiber initialization, Uber Fx graph composition, route mounting
├── internal/
│   ├── config/config.go  # Environment configuration (MongoURI, DBName, JWTSecret, Port, FSPath)
│   ├── database/database.go # MongoDB client lifecycle hook & DB instance creation
│   ├── middleware/
│   │   ├── auth.go       # AuthMiddleware: verifies Authorization Bearer <token>, extracts user_id
│   │   └── cors.go        # Permissive CORS headers for Mobile & Web clients
│   └── features/
│       ├── user/          # User/Auth controller, domain models, Mongo repository, JWT issuance
│       ├── job/           # Job CRUD, text indexing, salary & city filtering
│       ├── application/   # Application submission, duplicate prevention, recruiter review
│       ├── chat/          # Hub connection pool, REST endpoints, WebSocket upgrade handler
│       ├── network/       # P2P connection requests, acceptance, and status tracking
│       ├── mentorship/    # Expert profiles, categorization, booking slots
│       ├── event/         # Event creation, attendee list tracking, registrations
│       ├── skill/         # Skill tags repository & category grouping
│       ├── city/          # Cities master collection & search
│       └── file/          # Multipart upload handling & local/S3 file streaming
```

Database Entity Mapping

Entity	Mongo Collection	Key Fields	Flutter Model
User	users	_id, mobile, email, password, name, headline, about, skills, education, experience, projects, resume_url, wallet_balance	UserProfile (user_profile.dart)
Job	jobs	_id, recruiter_id, title, description, company, city_id, city_name, salary_min, salary_max, job_type, status, requirements	Job (job.dart)
Application	applications	_id, job_id, candidate_id, recruiter_id, status, cover_letter, created_at	ApplicationItem (application.dart)
Conversation	conversations	_id, participants (array of user IDs), last_message_id, last_message, updated_at	ConversationItem (conversation.dart)
Message	messages	_id, conversation_id, sender_id, content, is_read, created_at	ChatMessage (chat_message.dart)
Connection	network_connections	_id, sender_id, receiver_id, status (pending, connected, ignored), created_at	ConnectionRequestItem (connection_request.dart)
Event	events	_id, title, organizer, description, date, time, location, image_url, participants	EventItem (event.dart)
Mentorship	mentorships	_id, expert_id, title, description, category, duration, price, rating	ExpertItem (expert_profile.dart)

8. State Management (Riverpod) Guide

Key Providers & Implementation Patterns

1. Notifier Providers (Complex State + Methods)

- authProvider (NotifierProvider<AuthNotifier, AuthState>):
 - Manages current candidate session, profile mutations, resume uploads, and login/logout lifecycles.
- jobsProvider (NotifierProvider<JobsNotifier, JobsState>):
 - Maintains search filters, active pagination page, saved job IDs, and asynchronous job results.

2. FutureProviders (Cached Async Operations)

- conversationsProvider (FutureProvider<List<ConversationItem>>): Fetches the user's active conversations. Invalidated on sending a new message.
- myApplicationsProvider (FutureProvider<List<ApplicationItem>>): Fetches candidate application history.
- myInterviewsProvider (FutureProvider<List<InterviewItem>>): Fetches scheduled interviews.
- citiesProvider (FutureProvider<List<City>>): Loads pan-India city list for location selection.
- eventsProvider (FutureProvider<List<EventItem>>): Loads upcoming webinars and job fairs.
- expertsProvider (FutureProvider<List<ExpertItem>>): Loads industry mentors.

3. StreamProviders (Live Socket Streams)

- `websocketStatusStreamProvider ( StreamProvider<WebSocketStatus> )`: Streams real-time connection status (`connected` , `connecting` , `disconnected` , `error`) to UI badges.

4. ChangeNotifierProvider.family (Scoped Room State)

- `chatMessagesProvider ( ChangeNotifierProvider.family<ChatMessagesNotifier, String> )`: Scoped per `conversationId` . Automatically hooks into `ChatWebSocketService` upon construction, downloads message history, listens to live WebSocket messages, and disposes subscriptions on room exit.

9. Error Handling & Resilience Architecture

```

HTTP Network Request
  ↓ (Network drops / 401 Unauthorized / 500 Server Error)
Dio Client Interceptor [lib/core/network/api_client.dart]
  ├── Status = 401 Unauthorized:
  │   ├── LocalStorage.clearSession()
  │   └── Forces AuthNotifier reset → Redirects to Login
  ├── Extract Backend Error Message:
  │   └── data['error'] ?? data['message'] ?? 'Connection timed out'
  └── Re-throws standardized DioException with human-readable .error message
↓
Repository Layer:
  └── Catches DioException → Returns clean empty defaults or re-throws domain exceptions
↓
Notifier / State Layer:
  └── Sets state = state.copyWith(isLoading: false, errorMessage: errorMsg)
↓
Presentation Layer:
  ├── Renders custom Error View with "Retry" button
  └── Shows Snackbar / Toast with exact error description
  
```

10. Beginner Developer Walkthrough: "How the App Works"

1. **User Opens App:** `main.dart` initializes local preferences and mounts `KaamMilegaApp` . The router launches `/splash` . `SplashScreen` checks `LocalStorage` for a stored JWT token. If found, it fetches the user's live profile in the background; otherwise, it marks the session as guest and navigates to `/home` .
2. **Home Screen & Browsing:** `HomeScreen` renders within `MainNavigationShell` . It shows the instant gig banner, quick action circular icons, popular categories, and recommended jobs from `jobsProvider` .
3. **Searching & Filtering Jobs:** The user taps the Jobs tab or a category. `JobsScreen` watches `jobsProvider` . Typing in the search bar or opening the `FilterModal` invokes `JobsNotifier` methods. The notifier converts filter options into query parameters (`/api/jobs?city_ids=...&salary_min=...`), calls `JobRepository` , and updates `JobsState` .
4. **Applying for a Job:** On `JobDetailScreen` , tapping "Apply Now" triggers `ApplyModal` . If the user is unauthenticated, an `AuthPromptDialog` pops up. Once authenticated, `ApplicationRepository.applyToJob` sends a POST request with the candidate's ID and cover letter.
5. **Real-Time Chat:** When opening a conversation, `ChatMessagesNotifier` fetches history via REST (`GET /api/chats/:id/messages`) and opens a WebSocket to `wss://api.kaammilega.com/api/ws/chats` . When the recruiter responds, the Go backend pushes the message down the socket, and the chat screen updates in real time.
6. **Editing Profile & Uploading Documents:** `ProfileScreen` allows updating bio, education, experience, and portfolio. Tapping "Upload Resume" opens the file picker, sends the file as multipart data to `/api/files/upload` , receives an absolute URL, and patches the candidate record via `/api/user/profile` .

11. Developer Learning Path: "What Should I Read First?"

Step	File Path	Why Read It?	Next File
1	<code>pubspec.yaml</code>	Understand external libraries, Riverpod version, Dio, and GoRouter.	<code>lib/main.dart</code>
2	<code>lib/main.dart</code>	Inspect initialization order and root <code>ProviderScope</code> .	<code>lib/app/router.dart</code>
3	<code>lib/app/router.dart</code>	Learn all URL paths, parameter extractions, and route destinations.	<code>lib/core/network/api_client.dart</code>
4	<code>lib/core/network/api_client.dart</code>	Understand how Bearer tokens are attached and 401s handled.	<code>lib/features/auth/providers/auth_provider.dart</code>
5	<code>lib/features/auth/providers/auth_provider.dart</code>	Master login, OTP, registration, and user profile state.	<code>lib/features/jobs/providers/jobs_provider.dart</code>
6	<code>lib/features/jobs/providers/jobs_provider.dart</code>	Understand searching, filtering, and pagination mechanics.	<code>lib/features/chat/services/chat_websocket_service.dart</code>
7	<code>lib/features/chat/services/chat_websocket_service.dart</code>	Understand live bi-directional messaging and reconnection logic.	<code>lib/features/navigation/presentation/main_navigation_shell.dart</code>
8	<code>lib/features/navigation/presentation/main_navigation_shell.dart</code>	Understand the 5-tab shell structure and tab switching.	<code>km-backend/cmd/main.go</code>
9	<code>km-backend/cmd/main.go</code>	Inspect the Go backend Uber Fx dependency graph and route registrations.	<code>km-backend/internal/features/user/api.go</code>

12. Known Boundaries & Backend Integration Points

1. Payment Gateway Integration (`/api/wallet/add-money` & `/api/wallet/withdraw`):

- The Flutter `WalletRepository` contains robust exception handling (`WalletApiException.isBackendPending = true`) expecting Razorpay / Cashfree order creation endpoints. When deploying the gateway service in `km-backend`, wire the handler to `/api/wallet/add-money`.

2. Instant Gig Worker Dispatch Engine (`/instant-work`):

- The UI currently submits instant work requests and triggers automated matching feedback. The underlying matching algorithm connects to the `/api/jobs` spot-dispatch service.

3. Admin Moderation Interfaces:

- Handlers for `/api/admin/jobs`, `/api/admin/users`, and `/api/admin/companies` exist in `km-backend` and are used by the administrative portal; the mobile client operates under role `'user'`.

Documentation compiled and verified against the live KaamMilega Flutter codebase (`KaamMilegaApp`) and Go backend (`km-backend`).